Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет Информационных технологий и программирования

Лабораторная работа № 5

Анализ кода на уязвимости (SAST & DAST)

Выполнил:

Ягодин Захар Сергеевич, M3401

Санкт-Петербург
2026

Цель работы:

Научиться применять инструменты статического (SAST) и динамического (DAST) анализа безопа сности кода, а также интерпретировать их результаты.

Задание 1. Статический анализ кода (SAST)

1. Установите Semgrep на свою систему.

2. Выберите открытый исходный код (например, уязвимое приложение из OWASP WebGoat или Juice Shop).

https://github.com/iphyviktor/WebGoat.NETCore

3. Проведите анализ кода с помощью SAST-инструмента.

```
PS C:\Users\HuTao1Love\Obsidian\hw\inf-sec\lab5\WebGoat.NETCore> semgrep scan --config=p/owasp-top-ten --json > semgrep-report.json


┌─────────────┐
│ Scan Status │
└─────────────┘
  Scanning 201 files tracked by git with 544 Code rules:

  Language      Rules   Files          Origin      Rules
  ─────────────────────────────        ────────────────────
  <multilang>       6     201          Community     544
  csharp           27      54
  json              3       6
  js               65       2
  yaml             19       2
  bash              1       2
  dockerfile        4       1

  ────────────────────────────────────────── 100% 0:00:00


┌──────────────┐
│ Scan Summary │
└──────────────┘
✅ Scan completed successfully.
  • Findings: 24 (24 blocking)
  • Rules run: 125
  • Targets scanned: 201
  • Parsed lines: ~100.0%
  • Scan skipped:
    ◦ Files larger than  files 1.0 MB: 1
    ◦ Files matching .semgrepignore patterns: 28
  • Scan was limited to files tracked by git
  • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 125 rules on 201 files: 24 findings.
PS C:\Users\HuTao1Love\Obsidian\hw\inf-sec\lab5\WebGoat.NETCore>
```

```
PS C:\Users\HuTao1Love\Obsidian\hw\inf-sec\lab5\WebGoat.NETCore> $data = Get-Content semgrep-report.json | ConvertFrom-Json
PS C:\Users\HuTao1Love\Obsidian\hw\inf-sec\lab5\WebGoat.NETCore> $data.results | Select-Object check_id, path, start, end | Format-Table -AutoSize

check_id                                                                                  path                                        start                          end
--------                                                                                  ----                                        -----                          ---
dockerfile.security.missing-user-entrypoint.missing-user-entrypoint                       Dockerfile                                  @{line=18; col=1; offset=570}  @{line=18; col=41; offset=610}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\AccountController.cs @{line=29; col=20; offset=972}  @{line=32; col=15; offset=1062}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\AccountController.cs @{line=155; col=24; offset=5211} @{line=155; col=35; offset=5222}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\AccountController.cs @{line=172; col=20; offset=5947} @{line=172; col=31; offset=5958}
csharp.dotnet.security.audit.missing-or-broken-authorization.missing-or-broken-authorization WebGoat.NET\Controllers\BlogController.cs  @{line=9; col=6; offset=191}   @{line=60; col=6; offset=1964}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\BlogController.cs  @{line=29; col=20; offset=951}  @{line=29; col=68; offset=999}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\BlogController.cs  @{line=57; col=20; offset=1928} @{line=57; col=35; offset=1943}
csharp.dotnet.security.audit.missing-or-broken-authorization.missing-or-broken-authorization WebGoat.NET\Controllers\CartController.cs  @{line=8; col=6; offset=148}   @{line=90; col=6; offset=2782}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\CartController.cs  @{line=75; col=28; offset=2247} @{line=75; col=121; offset=2340}
csharp.dotnet.security.audit.missing-or-broken-authorization.missing-or-broken-authorization WebGoat.NET\Controllers\CheckoutController.cs @{line=16; col=5; offset=399}  @{line=243; col=6; offset=8870}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\CheckoutController.cs @{line=85; col=24; offset=3291} @{line=85; col=35; offset=3302}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\CheckoutController.cs @{line=108; col=24; offset=4129} @{line=108; col=36; offset=4141}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\CheckoutController.cs @{line=116; col=24; offset=4462} @{line=116; col=36; offset=4474}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\CheckoutController.cs @{line=186; col=20; offset=7036} @{line=186; col=31; offset=7047}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\CheckoutController.cs @{line=214; col=20; offset=7905} @{line=214; col=31; offset=7916}
csharp.dotnet.security.audit.missing-or-broken-authorization.missing-or-broken-authorization WebGoat.NET\Controllers\ProductController.cs @{line=12; col=6; offset=279}  @{line=148; col=6; offset=5316}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\ProductController.cs @{line=42; col=20; offset=1671} @{line=48; col=15; offset=1946}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\ProductController.cs @{line=72; col=20; offset=2780} @{line=72; col=31; offset=2791}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\ProductController.cs @{line=104; col=24; offset=3724} @{line=110; col=19; offset=4042}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\ProductController.cs @{line=118; col=20; offset=4211} @{line=123; col=15; offset=4468}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\ProductController.cs @{line=131; col=20; offset=4692} @{line=136; col=15; offset=4919}
csharp.dotnet.security.audit.mass-assignment.mass-assignment                              WebGoat.NET\Controllers\StatusCodeController.cs @{line=20; col=24; offset=509}  @{line=20; col=79; offset=564}
csharp.lang.security.sqli.csharp-sqli.csharp-sqli                                         WebGoat.NET\Data\OrderRepository.cs        @{line=49; col=39; offset=2101} @{line=49; col=42; offset=2104}
csharp.lang.security.sqli.csharp-sqli.csharp-sqli                                         WebGoat.NET\Data\OrderRepository.cs        @{line=80; col=39; offset=3507} @{line=80; col=42; offset=3510}
```

4. Проанализируйте отчёт (Какие уязвимости были найдены? Как их можно исправить?)

Название уязвимости	Проблема	Решение
dockerfile.security.missing-user-entrypoint.missing-user-entrypoint	контейнер запускается от root (нет директивы USER)	Добавить пользователя в

Название уязвимости	Проблема	Решение
		Dockerfile и запускать от него
csharp.dotnet.security.audit.mass-assignment	Массовое присвоение свойств объекта из пользовательского ввода	Использовать DTO и явное присвоение
csharp.dotnet.security.audit.missing-or-broken-authorization.missing-or-broken-authorization	Эндпоинты без проверки прав	Добавить атрибуты авторизации
csharp.lang.security.sqli.csharp-sqli	SQL инъекция	Использование параметров или ORM

5. Создайте Pull Request с исправлениями найденных уязвимостей.

Для примера сделал PR с исправлением SQL инъекции и докерфайла - https://github.com/HuTao1Love/WebGoat.NETCore/pull/1

6. Приложите выгрузку отчета SAST к отчету по лабораторной работе

Pastebin - https://pastebin.com/fVZjazXx

Пароль - `iJMqaexkUb`

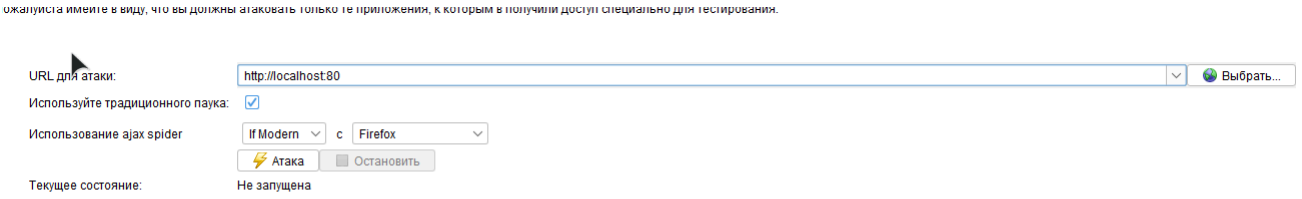
Задание 2 Динамический анализ безопасности (DAST)

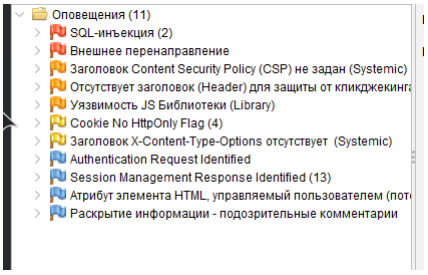
1. Установите OWASP ZAP или аналогичный инструмент для DAST.

2. Запустите тестовое уязвимое приложение из 4-й лабораторной работы (либо OWASP WebGoat, Juice Shop) в контейнере Docker.

```
PS C:\Users\HuTao1Love\RiderProjects\WebGoat.NETCore> docker run   80:8080 myapp
warn: Microsoft.AspNetCore.DataProtection.Repositories.FileSystemXmlRepository[60]
      Storing keys in a directory '/root/.aspnet/DataProtection-Keys' that may not be persisted outside of the container. Protected data will be unavailable when container is destroyed. For more information go to https://aka.ms/aspnet/dataprotectionwarning
warn: Microsoft.AspNetCore.DataProtection.KeyManagement.XmlKeyManager[35]
      No XML encryptor configured. Key {11728fd7-7406-49ec-a39a-7bf4496f54c2} may be persisted to storage in unencrypted form.
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://[::]:8080
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /app
```

3. Проведите автоматизированное сканирование уязвимостей с помощью DAST-инструмента.





4. Проанализируйте отчёт сканирования:

- Какие типы уязвимостей были обнаружены?
- Можно ли воспроизвести их вручную?
- Как их можно исправить?

Название уязвимости	Проблема	Решение	Вручную?
SQL инъекция	Возможность внедрения SQL-кода через параметры запроса или формы, что может привести к утечке/изменению данных	Использовать подготовленные выражения (Prepared Statements), ORM, строгую валидацию и типизацию входных данных	Да
Внешнее перенаправление	Пользователь может быть перенаправлен на вредоносный сайт через параметр URL	Проверять и разрешать только доверенные URL (белый список), не использовать прямую передачу URL из запроса	Изменить параметр редиректа (`?redirect=http://...) и проверить переход
Не задан заголовок Content Security Policy (CSP)	Повышенный риск XSS и загрузки вредоносных скриптов	Добавить заголовок `Content-Security-Policy` с ограничением источников (`default-src 'self'`)	Проверить заголовки ответа сервера через DevTools → Network
Отсутствует заголовок защиты от кликджекинга	Страницу можно встроить во фрейм и обманом заставить пользователя выполнить действие	Добавить `X-Frame-Options: DENY` или `CSP: frame-ancestors 'none'`	Попытаться встроить страницу в фрейм
Уязвимость JavaScript-библиотеки	Используется версия библиотеки с известными CVE	Обновить библиотеку до актуальной версии, удалить неиспользуемые зависимости	Проверить версию библиотеки и сопоставить с публичными уязвимостями
Cookie No HttpOnly Flag	Cookies доступны через JavaScript → риск кражи сессии при XSS	Установить флаги `HttpOnly` , `Secure` , `SameSite`	В консоли браузера выполнить `document.cookie` и проверить доступность
Отсутствует заголовок X-Content-Type-Options	Возможен MIME-sniffing и выполнение вредоносного контента	Добавить заголовок `X-Content-Type-Options: nosniff`	Проверить наличие заголовка в ответе сервера
Session Management Response Identified	Возможные слабости в управлении сессией (фиксация, долгоживущие токены)	Генерировать новые ID при логине, ограничить время жизни сессии, использовать Secure и HttpOnly	Проверить, меняется ли session ID после авторизации
Атрибут HTML, управляемый	Возможен XSS через вставку данных в	Экранировать вывод, использовать	Ввести `<img src=x onerror=alert(1)>` и проверить выполнение

Название уязвимости	Проблема	Решение	Вручную?
пользователем (потенциально)	атрибуты (`onerror` , `onclick`)	безопасные шаблонизаторы	
Раскрытие информации — подозрительные комментарии	В коде могут содержаться данные о структуре системы, путях, логике	Удалить debug-комментарии, отключить подробные ошибки на production	Просмотреть исходный код страницы (View Source)

5. Приложите выгрузку отчета DAST к отчету по лабораторной работе

Дополнительные вопросы

1. Какие типы уязвимостей лучше всего выявляются SAST?
   SAST - для поиска ошибок в структуре кода:
   - SQL-инъекции и XSS: прослеживание пути данных от ввода до исполнения.
   - Хардкод кредов: поиск паролей, ключей API и токенов, зашитых в код.
   - Использование небезопасных функций: например, `eval()` в JS/Python.
   - Ошибки криптографии: использование слабых алгоритмов шифрования (MD5, SHA1).
   - Переполнение буфера: особенно актуально для компилируемых языков (C/C++).
2. Какие типы уязвимостей можно найти только при помощи DAST?
   - Ошибки конфигурации сервера: неправильные заголовки HTTP, включенные методы (TRACE/TRACK).
   - Проблемы с сессиями: небезопасные Cookie (отсутствие флагов HttpOnly/Secure).
   - Ошибки логики аутентификации: уязвимости, возникающие только при взаимодействии компонентов.
   - Проблемы инфраструктуры: уязвимости в сторонних библиотеках среды выполнения или веб-сервера.
3. Какой анализ проводить первым: статический или динамический? Почему?
   SAST:
   - Экономия: ошибки исправляются дешевле на этапе написания кода.
   - Скорость: SAST можно запускать сразу после написания функции, не дожидаясь сборки и развертывания приложения.
   - Покрытие: SAST проверяет все ветки кода, включая те, которые трудно вызвать в работающем интерфейсе.
4. В каких случаях SAST даёт ложные срабатывания?
   - Особенно актуально для динамически типизированных языков
   - Не понимает контекст: например, не видит, что данные уже очищены внешним фреймворком или WAF.
   - Сложная логика: теряет путь переменной в цепочке из множества вызовов функций.
   - Универсальные правила: слишком строгие паттерны, которые не учитывают специфику конкретной бизнес-логики.
5. Как избежать ошибок при настройке DAST-сканирования?
   - Настройка аутентификации
   - Ограничение области: указать URL-адреса, чтобы сканер не ушел на сторонние ресурсы (например, ссылки на соцсети).

- Безопасность данных: проводить сканирование в staging-среде, так как DAST может заполнять БД мусором или случайно удалить записи через формы.
- Исключение деструктивных действий: отключить проверку функций удаления или сброса пароля.

6. Какие ограничения есть у SAST-сканирования по сравнению с DAST?
   - Отсутствие контекста среды: SAST не знает, на какой ОС и с какими настройками сервера работает код.
   - Зависимость от языка: SAST требует глубокой поддержки синтаксиса конкретного языка программирования.
   - Слепота к интеграциям: SAST плохо видит проблемы, возникающие на стыке приложения и базы данных или внешних API.
7. Как защититься от уязвимостей, которые не детектируются ни SAST, ни DAST?
   - Анализ зависимостей (библиотек) + локальное зеркало (nuget/pypi/etc)
   - Ручной пентест
   - Code Review
   - WAF (Web Application Firewall)
8. Какие open-source инструменты используются для SAST и DAST, и в чём их отличия?
   1. SAST
      - Semgrep
      - SonarQube
      - GitHub CodeQL
      - Линтер, встроенный в IDE/компилятор (например, анализаторы Roslyn в dotnet publish) - не совсем SAST, но...
   2. DAST
      - OWASP ZAP (Zed Attack Proxy)
      - Nuclei
9. Как часто следует проводить статический и динамический анализ кода в CI/CD?
   - SAST - при каждом пуше или MPe
   - DAST - при каждом деплое на тестовую среду или по расписанию
10. В чём разница между интерпретируемыми и компилируемыми языками с точки зрения анализ а уязвимостей?
    - Компилируемые (C++, C#, JAVA): SAST ищет ошибки управления памятью (утечки, переполнения). Для анализа часто требуется сначала успешно собрать проект.
    - Интерпретируемые (Python, JS, PHP): SAST работает проще (анализ скриптов), но выше риск ложных срабатываний. Ошибки типизации выявляются сложнее без запуска.
11. Как можно использовать результаты DAST-сканирования для улучшения защиты веб-приложения?
    - Виртуальный патчинг: настройка правил WAF на основе найденных уязвимостей, пока разработчики готовят исправление.
    - Приоритизация: если DAST подтвердил уязвимость, найденную SAST, её приоритет становится критическим (так как она точно эксплуатируема).
12. Почему важно учитывать контекст уязвимости при интерпретации отчёта SAST?
    Например, проблемы в тестовом скрипте - низкий риск, а в проде - крит инцидент (хардкоды,

sql инъекции, etc)

13. Можно ли полностью автоматизировать анализ безопасности кода, или требуется ручная про верка?
    Инструменты - только для поиск известных паттернов
    Человек - для отсеивания ложных срабатывания, анализа бизнес-логики, цепочек уязвимостей